Automatic Code-to-Architecture Mapping with Graph Neural Networks

Rakhshanda Jabeen*†, Morgan Ericsson†, Jonas Nordqvist†, and Anna Wingkvist†
*Electrolux Professional AB, Ljungby, Sweden
†Linnaeus University, Växjö, Sweden
Email: {rakhshanda.jabeen | morgan.ericsson | jonas.nordqvist | anna.wingkvist}@lnu.se

***Abstract*—Architectural technical debt arises when a system's implementation drifts from its intended modular decomposition, while the mapping between source files and architectural modules is missing, incomplete, or costly to maintain manually. We formulate code-to-architecture mapping as semi-supervised node classification over a directed multi-relational dependency graph, with files as nodes, typed dependencies as edges, and lexical features derived from code identifiers and directory layout. Starting from a small labeled sample, an iterative self-training procedure expands the mapping to the remaining files. We compare a graph convolutional network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP), using identical features and training procedures for the learned models, against a Naive Bayes reference. Across 100 runs on ten open-source Java and C/C++ systems, all learned models substantially outperform the reference. GAT achieves the highest overall mean macro and micro F1, attaining the highest or tied-highest micro F1 on nine systems, whereas MLP remains competitive or superior on several small systems with weak connectivity or weak modular cohesion. On Chrome, the largest system with 18,343 files, GAT and GCN exceed MLP by 22 and 19 macro-F1 points, respectively. The graph also improves file-level repeatability: persistently wrong files remain below 5% for both GNNs on every system, compared with up to 78% for MLP. Connectivity and homophily determine where message passing helps, while multi-relational aggregation outperforms a single merged edge type on every system for both GNNs. The remaining persistent errors mainly occur when dependency neighborhoods contradict a file's true module or provide diffuse or insufficient structural evidence.**

I. Introduction

Software architecture defines a system's high-level structure and the dependencies among its principal elements, and it provides the basis for reasoning about maintainability, security, and other quality attributes [1]. That reasoning requires the implementation to stay aligned with the intended design. When it does not, the gap is architectural technical debt [2]: cheap to introduce, but diffuse and relational rather than local to a single file, and therefore expensive to detect and repay once it accumulates. Large language models (LLMs) are now widely used to generate or assist with code commits, compounding this problem because they produce code that satisfies the immediate prompt without visibility into the system's module structure [3]. Studies of AI-generated code report non-trivial clone-like repetition across tasks, suggesting that these models reproduce learned patterns rather than reusing project-specific abstractions [4]. As this code enters the system unchecked, the implementation drifts from the intended design, causing architectural erosion [5].

Detecting this drift requires comparing the intended architecture against the implemented one. Static Architecture Conformance Checking (SACC) addresses this comparison, and reflexion modeling is a SACC technique that classifies relations between the intended and implemented architectures as convergent, divergent, or absent [6, 7]. However, reflexion modeling depends on a mapping between source code entities and architectural modules, and such mappings are often missing, incomplete, or expensive to maintain manually [8]. Even for medium-sized systems, recovering or maintaining the architectural decomposition can require substantial expert effort [9]. Automated code-to-architecture (C2A) mapping is therefore an important prerequisite for practical conformance checking.

Prior work on C2A mapping has explored supervised and recommendation-based approaches. Supervised approaches compute *attraction values* that quantify the association strength between source code entities and architectural modules [10, 11]. Olsson et al. [12] proposed a semi-supervised Naive Bayes mapper that combines syntactic and semantic information to assign source code entities to architectural modules. InMap [13] took a recommendation-based view instead, using information retrieval over the codebase to suggest candidate mappings with minimal architectural documentation and no training data.

A separate line of work targets architecture recovery, where the decomposition itself must be discovered rather than assumed. Recent recovery approaches include graph neural networks (GNNs) combined with contrastive learning and autoencoders [14, 15], along with language models applied directly to source code for recovering design structures [16], for agent-based recovery of large legacy systems [17], and for extracting architectural entities to support traceability [18]. The distinction from C2A mapping matters here: the module set is unknown in recovery, whereas in mapping it is given by the intended architecture, and the task is to assign each file to one of those modules. It also matters for evaluation, since many recovery benchmarks are open source, and a decomposition reproduced by a pretrained model may reflect code understanding, prior exposure to the system, or both. Mapping methods that learn from a system's own structure

and can be evaluated against a known ground truth therefore remain important.

In this research, we study C2A mapping[1] as semi-supervised node classification over a multi-relational directed dependency graph. Source files are nodes, typed static dependencies between them are edges, and node features encode code identifiers via Word2Vec embeddings together with directory context. A small set of labeled files is expanded into a complete mapping through iterative self-training. We compare three encoders under the same features and training procedure: a graph convolution network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP). Holding the feature representation, the classifier head, and the self-training loop fixed is intended to isolate what the dependency graph contributes beyond the node features alone, though the encoders themselves differ in architecture beyond the presence or absence of message passing.

We evaluate the approach on ten open-source systems written in Java and C/C++, reporting both mapping accuracy and computational cost. This paper extends our earlier study [19] in five respects: it broadens the benchmark by adding C/C++ systems; replaces the earlier bag-of-words representation of code identifiers with Word2Vec embeddings; adds GAT to assess the effect of attention-based aggregation on the mapping task; introduces an MLP baseline to isolate the contribution of dependency information; and corrects a label-assignment defect in the earlier self-training pipeline. We further evaluate the corrected self-training procedure and conduct new ablation studies of model depth and edge-aggregation direction.

Our study is guided by the following research questions:

- **RQ1** How effectively does GNN-based C2A mapping perform relative to non-graph baselines, and how does it scale?
- **RQ2** What does the dependency graph contribute beyond textual features under an identical training procedure, and which system properties govern where it helps?
- **RQ3** How sensitive is the approach to modeling dependency types separately, aggregation depth, and edge direction?

II. Related Work

Tzerpos and Holt [20] introduced the orphan adoption problem, in which source code entities that are not yet part of the documented architecture are incrementally assigned to modules. Christl et al. [10] built on this idea with the Human-Guided Clustering Method (HuGMe), a semi-automatic approach that recommends mappings through attraction functions and leaves the final decision to the architect. Bittencourt et al. [11] studied attraction functions derived from information retrieval, where an entity is assigned to the module whose textual content it most resembles. These approaches rely on two signals that persist in later work: lexical similarity between an entity and a module and dependency proximity to entities that are already mapped, but they combine them through fixed, manually specified rules.

Subsequent work replaced these rules with learned models. Olsson et al. [12] proposed a semi-supervised Naive Bayes mapper that combines lexical information from source code with dependency-derived information through Concrete Dependency Abstraction, and reported that 15–20% of the entities must be mapped beforehand to reach acceptable performance. Florean et al. [21] compared text classifiers for the same task and showed that both the classifier and the text representation affect mapping quality. InMap [13] instead ranks candidate mappings interactively, using natural language module descriptions together with source code, which removes the need for a pre-mapped training set. In all of these, dependencies enter the model only after being flattened into features attached to individual entities, so the relational structure of the system is consumed rather than represented.

Language models have recently been applied to the same task. Rukmono et al. [22] use chain-of-thought prompting with GPT-4 to deductively assign implementation units to a predefined reference architecture. Johansson et al. [23] evaluate language model embeddings alongside TF-IDF and GloVe as input to method-level classifiers, and in follow-up work attribute much of the variation across projects to lexical overlap and skew in the module size distribution rather than to the choice of model [24]. A parallel line applies language models to architecture recovery, where the module set is inferred rather than given [16, 17]. Pretrained models capture semantic cues that engineered text features miss, but their input remains predominantly textual, so typed dependencies stay implicit, and each entity is reasoned about largely in isolation. Prompting also scales poorly, since context limits and per-entity inference cost grow with the number of files.

Graph representation learning uses dependencies as structure rather than as features. NEGAR [25] showed that random-walk embeddings over the dependency graph, followed by clustering, recover meaningful architectural structure. GAER [14] extended this to heterogeneous graphs, comparing GCN and GAT encoders trained within a graph autoencoder and modeling each dependency type as a separate relation, and [15] reports related results using contrastive objectives. Both are unsupervised recovery methods, so the module set is discovered by clustering rather than fixed in advance as it is in mapping. Our earlier work [19] took this step and evaluated GCN-based models on homogeneous and dependency-type-aware graphs, showing that a small labeled set can be expanded through self-training. That study was limited to Java systems of moderate size, used a single graph-based encoder family, and did not isolate how much of the observed performance came from the dependency graph as opposed to the node features.

The present work addresses these limitations. We evaluate GNN encoders together with a graph-free neural baseline that receives identical features and an identical self-training loop, so any difference between them is consistent with the contribution of the dependency graph rather than of representation

[1] Source code is available at https://github.com/sse-lnu/GCN-SAEroCon25.

or optimization.

III. Methodology

This section presents our approach to C2A mapping as semi-supervised node classification over a directed multi-relational dependency graph. We first formulate the mapping task, then describe the graph and feature construction, introduce the encoders compared in this study, and finally the self-training procedure used to grow an initial labeling into a full mapping. Figure 1 illustrates the heterogeneous GNN encoder at the core of this pipeline.

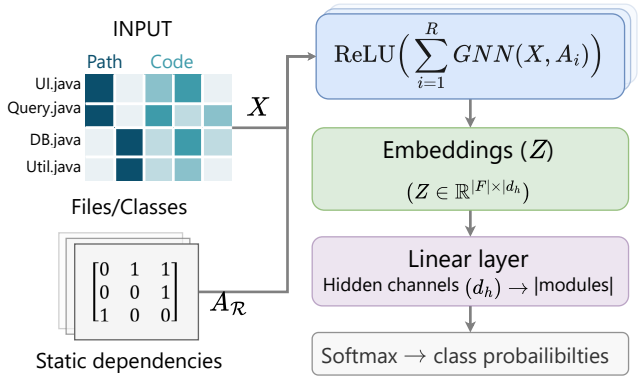


Fig. 1: Overview of the heterogeneous GNN encoder. Node features X (file-path indicator + Word2Vec identifiers) and relation-specific adjacency matrices $A_{\mathcal{R}}$ are passed through a relation-specific encoder (GCN or GAT), whose outputs are summed and passed through a non-linear activation function (ReLU) to produce embeddings $Z \in \mathbb{R}^{|\mathcal{F}| \times d_h}$, where d_h is the number of hidden channels. A linear layer maps Z to per-module logits, and a final softmax produces the class probabilities used for classification and pseudo-labeling.

A. Problem Formulation

Given a software system with a set of files $\mathcal{F}$ and a fixed set of architectural modules $\mathcal{C}$, a C2A mapping assigns each file $i \in \mathcal{F}$ to a module $y_i \in \mathcal{C}$. A small subset $L \subset \mathcal{F}$ is labeled with its ground-truth module; the remaining files $U = \mathcal{F} \setminus L$ are unlabeled, and the task is to recover the assignment for every file in U. We consider a transductive setting in which the lexical node features and dependency structure of all files are available during training, while module labels are available only for files in L.

Two kinds of sources of information are available for each file. The first is intrinsic and consists of what the file itself declares (identifiers) and where it sits in the source tree (directory structure), and it is available for every file regardless of how the file is connected to the rest of the system. The second is relational: the representations of its neighbors in the dependency graph, propagated through message passing and available only where the file has neighbors. We frame the task as representation learning so the two sources can be treated separately: every encoder maps a file i to an embedding z_i, and a shared classification head maps z_i to a module, so the encoders differ in what evidence enters the encoder to generate z_i and not in how it is used.

B. Graph Construction

We represent a software system as a heterogeneous graph $G = (X, A_{\mathcal{R}})$ over its files, following the construction introduced for unsupervised recovery in GAER [14]. Code identifiers and directory paths are file-level properties and are encoded in the feature matrix X. Static dependencies are file-to-file relations, each belonging to a dependency type r drawn from the set of relation types $\mathcal{R}$, and encoded as edges in the multi-relational adjacency $A_{\mathcal{R}} = \{A^{(r)}\}_{r \in \mathcal{R}}$.

1) Node Features : The intrinsic evidence for a file comes from two conventions developers follow when they write and organize code. Identifiers name the concepts a file works with, so class, method, and variable names carry the vocabulary of the domain the file serves and, with it, an indication of the responsibility it implements [26]. Directory paths record how developers chose to group files, and that grouping frequently tracks the subsystem decomposition the architecture describes [9]. The two are complementary where structure alone is ambiguous, since a widely used utility file may depend on code throughout the system while its name and its location still indicate what it is for. Both signals are available from source text alone, without language-specific resolution, which is what allows the same feature construction to serve the Java and the C/C++ systems in our benchmark.

We normalize code identifiers by splitting on *camelCase*, *PascalCase*, and *snake_case* boundaries and removing low-information tokens (stopwords, very short tokens, tokens appearing in over 90% of files), then train a 128-dimensional skip-gram Word2Vec [27] embedding over the remaining tokens for 20 epochs with a context window of 5. Let T_i denote the resulting sequence of normalized tokens for file i; its textual representation is the ℓ_2-normalized average of the Word2Vec embeddings of the tokens in T_i. File paths are tokenized separately and represented as a binary bag-of-words vector, which keeps each path component a distinct dimension so that a shared parent directory is visible to the model as a shared feature. For file i, the two are concatenated:

$$x_i = \left[\text{Binary(FilePath)}_i \,\middle\|\, \text{Word2Vec}(T_i)\right]. \tag{1}$$

Stacking x_i over all files gives the feature matrix X. Every encoder in this study receives the same X, so the intrinsic evidence available to each of them is identical by construction, and any difference between them originates in the relational evidence.

2) Static Dependencies (Edges) : Files that implement related functionality tend to depend on one another directly or through short dependency paths, which is what makes the dependency graph informative for architecture recovery [28]. Dependency types are not interchangeable in this respect, and call edges have been argued to indicate tighter coupling than type-usage edges [25, 29]. Collapsing them into one adjacency matrix forces the encoder to weight all of them alike, so we

keep one adjacency matrix per type and let the encoder learn a separate transformation for each. Section V-C evaluates this relation-typed design against a homogeneous graph for both GNN encoders.

We extract dependencies with the multilingual static-analysis tool *Depends* [30], which resolves the typed relations listed in Table I between code entities across our Java and C/C++ systems. Because the mapping task operates at file granularity, member-level dependencies such as a call between two methods are aggregated into edges between their enclosing files, and repeated dependencies of the same type between the same ordered pair collapse into a single edge, so $A^{(r)}$ records whether a relation holds and not how often it occurs.

TABLE I: Static dependencies used in the software graph.

Dependency	Description
`Annotation`	Annotation relation to an annotated entity.
`Call`	Function or method invocation.
`Cast`	Conversion of an entity to another type.
`Create`	Creation of an instance or program entity.
`Extend`	Inheritance or parent–child type relation.
`Implement`	A class implements an interface or abstraction.
`Import`	A file imports other classes, methods, or variables.
`Parameter`	Use of a type as a method or function parameter.
`Return`	Use of a type as a return type.
`Throw`	A type is used as an exception.
`Use`	Use or assignment of variables.

`Annotation` and `Throw` are not observed in the C/C++ systems.

A static dependency is asymmetric, and $A^{(r)}$ preserves that direction: an edge points from the file where it originates to the file it refers to. For file i under relation r, we distinguish its in-neighborhood, the files that depend on it, from its out-neighborhood, the files it depends on. A file with a large in-neighborhood and a small out-neighborhood provides functionality, the profile of an interface or utility, and its clients may be spread across many modules. A file with the opposite profile consumes functionality, like orchestration or entry-point code, and what it draws on says more about what it coordinates than about which module it belongs to. Which of the two an encoder aggregates over, or whether it discards the direction altogether and treats the edge as undirected, is therefore a modeling choice rather than a fixed property of the graph; Section III-C defines it as a parameter of message passing, and Section V-C tests its impact on mapping.

C. Representation Learning

We compare three encoders that operate on the same node features X. The graph-based models, GCN and GAT, use the relation-specific adjacency $A_{\mathcal{R}}$ to aggregate information from neighboring files, while the non-graph baseline uses X alone. For each file i, all models take $h_i^{(0)} = x_i$ as input, stack K layers to produce embeddings $z_i = h_i^{(K)}$, and feed z_i into a shared linear classification head followed by softmax to obtain module probabilities. The encoders differ only in the neighborhood they aggregate over and in how they weight neighboring nodes.

1) GNN Encoders: Both GNN encoders update the representation of each file by aggregating messages from its neighbors under each dependency type r, a mechanism known as message passing. The neighbors of file i under relation r form the set $\mathcal{N}_i^{r,\pi}$, where the propagation scheme π determines which side of the directed edge is aggregated over: forward, reverse, or undirected, as introduced in Section III-B2. A separate relation- and layer-specific kernel $W^{r,(k)}$ transforms each neighbor's message, so a call edge and an inheritance edge contribute through different learned transformations rather than being folded into a single adjacency. Each additional layer widens this neighborhood by one hop, so a K-layer encoder lets information reach a file from up to K dependency steps away. We add self-loops before aggregation so a file's own features are included in its update, which matters when its neighborhood is empty. Forward propagation is the default in Equations 2 and 3.

GCN: Following Kipf and Welling [31], a GCN layer updates each file's representation by averaging its neighbors' messages, weighted by the inverse square root of the endpoint degrees under each relation:

$$h_i^{(k+1)} = \mathrm{ReLU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \frac{1}{\sqrt{d_i^r d_j^r}} W^{r,(k)} h_j^{(k)} \Big), \qquad (2)$$

where $W^{r,(k)}$ is the relation- and layer-specific transformation, and d_i^r and d_j^r denote the number of neighbors of files i and j in $\mathcal{N}_i^{r,\pi}$ and $\mathcal{N}_j^{r,\pi}$, respectively, so the degree follows whichever propagation scheme π is in effect. ReLU [32] is the non-linear activation. The degree normalization prevents high-degree files, such as widely used base types, from dominating the aggregation.

GAT: Following Veličković et al. [33], GAT replaces fixed degree-based weighting with learned attention coefficients. For each relation r, the features of each neighboring file are projected using $W^{r,(k)}$, scored, and normalized with a softmax over $\mathcal{N}_i^{r,\pi}$ to obtain the attention weight α_{ij}^r. The layer update for file i is:

$$h_i^{(k+1)} = \mathrm{ReLU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \alpha_{ij}^r \, W^{r,(k)} h_j^{(k)} \Big). \qquad (3)$$

GCN weights messages according to node degrees, while GAT learns their importance from node features. This increases flexibility but introduces additional parameters. Consequently, changing the propagation scheme π changes only the set of aggregated neighbors in GCN, whereas in GAT it also changes the learned attention distribution over those neighbors.

2) Multi-layer Perceptron (MLP): To assess the contribution of graph-neighborhood information, we use an MLP that operates on x_i alone and does not aggregate messages from other files. For each file i, the layer update rule is defined as:

$$h_i^{(k+1)} = \mathrm{ReLU}\big(W^{(k)} h_i^{(k)} + b^{(k)}\big). \qquad (4)$$

Unlike the GNNs, whose representations incorporate information from neighboring files over multiple dependency hops, the MLP constructs each representation from the file's lexical

features only. Performance differences between the MLP and the GNNs are therefore consistent with the contribution of dependency-graph information, though they should not be interpreted as isolating graph aggregation alone, since GAT's attention mechanism introduces additional parameters that the other two encoders do not have.

D. Iterative Learning Framework

GCN and GAT receive the same dependency graph and node features, differing only in how they aggregate and weight messages from neighboring files; the MLP receives the node features alone and does not use the dependency graph. Despite these differences, all three models follow the same self-training procedure. Each produces a file embedding z_i, which the shared classification head maps to a probability over the architectural modules; the highest probability is the model's confidence in its predicted module for file i.

Training begins with a fixed set of labeled files L, while the remaining files form the unlabeled set U. After an initial warm-up phase on L, each round scores every file. Pseudo-labels whose current prediction no longer matches them are revised, and files from U whose confidence meets the threshold τ are promoted into the mapped set $\mathcal{M}$. The classifier is then retrained on $L \cup \mathcal{M}$. The procedure stops when no new confident files are found or when the maximum number of rounds is reached. Algorithm 1 summarizes this loop, identical across encoders except for the input representation and forward computation.

Algorithm 1: Iterative Mapping Algorithm.

Input: Input representation $\mathcal{I}$, files $\mathcal{F}$, labeled files L, confidence threshold τ, warm-up epochs E_w, self-training epochs E_s, max rounds R
Output: Trained classifier f, mapped files $\mathcal{M}$

$\mathcal{M} \leftarrow L$; $U \leftarrow \mathcal{F} \setminus L$;
Train f on $(\mathcal{I}, \mathcal{M})$ for E_w epochs;
for $k \leftarrow 1$ **to** R **do**
  Score all files in $\mathcal{F}$ with f using $\mathcal{I}$;
  Revise any pseudo-label in $\mathcal{M} \setminus L$ whose confidence $\geq \tau$ disagrees with f;
  $\mathcal{P} \leftarrow \{o \in U \mid \text{confidence}(o) \geq \tau\}$;
  if $\mathcal{P} = \emptyset$ **then**
    break;
  $\mathcal{M} \leftarrow \mathcal{M} \cup \mathcal{P}$; $U \leftarrow U \setminus \mathcal{P}$;
  Retrain f on $(\mathcal{I}, \mathcal{M})$ for E_s epochs;
return f, $\mathcal{M}$;

IV. Experiments

This section describes the experimental design used to answer the three research questions: the benchmark systems, configuration, baseline, and evaluation metrics.

A. Benchmark Systems

We rely on existing GT mappings rather than constructing new ones, since producing a reliable C2A mapping demands substantial architectural knowledge, manual effort, and often expert validation [9]. Table II lists the ten Java and C/C++ systems used in our evaluation. We also report file-level dependency homophily $\mathcal{H}$, the fraction of static dependency edges whose source and target files belong to the same ground-truth module, ranging from 0 (every dependency crosses modules) to 1 (every dependency stays within one). Each ordered pair of files with at least one static dependency between them contributes a single edge, regardless of how many individual dependencies or dependency types connect the pair.

TABLE II: Properties of the software systems. $\mathcal{C}$ denotes the number of modules in the GT architecture, edges are the static dependencies between files, and the average degree is computed as $\langle k \rangle = |E|/|\text{Files}|$.

	System	Version	LoC	$\mathcal{C}$	|Files|	$\lvert E\rvert$	$\mathcal{H}$	$\langle k \rangle$
Java	A.UML	r13713	62k	14	766	9 491	0.54	12.4
	C.Img	v1.0a2	18k	21	329	4 561	0.61	13.9
	JabRef	v3.7	59k	6	1 180	17 557	0.58	14.9
	Lucene	r1075001	36k	7	1 054	18 595	0.68	17.6
	SH-3D	r002382	35k	9	191	3 912	0.34	20.5
	T.Mates	v5.11	55k	15	778	17 818	0.33	22.9
C/C++	Bash	4.2	115k	14	292	1 222	0.60	4.2
	HDF	0e196f	153k	9	153	1 220	0.35	8.0
	HDC	46ff87	26k	11	207	1 117	0.63	5.4
	Chrome	svn-171054	11M	69	18 343	284 435	0.58	15.5

The Java systems are drawn from the same source as our earlier work [19], assembled by Olsson et al. [12] from systems previously studied at SAEroCon and by Brunet et al. [34] and Lenhard et al. [35], with mappings distributed through the s4rdm3x GitHub repository[2].

For Bash, we use the C2A mapping curated by Garcia et al. [9], produced by deriving an initial decomposition from documentation and source code and then validating it with system experts. The same line of work provides the GT mapping for Chrome, established through roughly two years of collaboration with the system's developers [36].

The GT mappings for HDF and HDC were developed in collaboration with industry partners and are included in the SARIF benchmark studied by Zhang et al. [26], distributed through the SARIF replication package[3].

B. Experimental Configuration

GCN, GAT, and the MLP all use two layers by default, and the effect of using a single message-passing layer is examined separately in Section V-C. GCN and GAT use 256 hidden channels, and the MLP uses 256 hidden units. GCN and GAT apply a dropout rate of 0.01 to regularize message aggregation; the MLP, which does not aggregate messages, uses no dropout. All three models are trained with the Adam optimizer at a learning rate of 0.001, minimizing cross-entropy loss over the architectural modules.

All models use the same iterative learning procedure: a warm-up phase of $E_w = 100$ epochs on the initial labeled set, followed by at most $R = 4$ self-training rounds of $E_s = 30$ epochs each. The initial labeled set contains 5% of a system's files, bounded between 30 and 200 files. The lower bound

[2]https://github.com/h0bb3/s4rdm3x
[3]https://github.com/anonymous2f4a9d/SARIF_FSE23

provides sufficient supervision in small systems, where 5% alone would yield too few labeled files, while the upper bound limits the manual labeling effort in large systems such as Chrome. After sampling, we add one randomly selected file from each module still missing from the initial labeled set, so every module has at least one labeled file before training. During self-training, only predictions with confidence above $\tau = 0.95$ are added as pseudo-labels. These settings are fixed across all models.

All experiments were run on a single HP ZBook Power G10 workstation with a 13th-generation Intel Core CPU, an NVIDIA RTX 3000 Ada Generation laptop GPU, and 32 GB of RAM; model training used the GPU.

C. Baseline: Naive Bayes

We compare against *NBAttract* [12] (NBA), a multinomial Naive Bayes classifier originally proposed for semi-supervised code-to-architecture mapping. NBA represents each entity using text features extracted from code identifiers and package names, combined with module-level Concrete Dependency Abstraction (CDA) sentences, document terms that encode cross-module structural relations, e.g., *A-Inherits-B* when an entity in module A inherits from one in module B. Olsson et al. report that NBA outperforms *CountAttract* [10] and the information retrieval methods *IRAttract* and *LSIAttract* [11] on the same benchmark [12], so we treat NBA rather than those earlier methods as the reference point for prior work. NBA was originally proposed and evaluated on Java systems only; we extend it to C/C++ systems in our benchmark and re-implement the algorithm in Python, following the original iterative procedure, including its 0.90 confidence threshold for adding pseudo-labels and the same initial labeled set used by the other methods. Self-training stops either when no remaining entity can be mapped with sufficient confidence or when all entities have been mapped. Code for our NBA re-implementation and C/C++ extension is available in the GitHub repository 1.

D. Performance Evaluation

Module distributions in the benchmark systems are highly imbalanced, with several modules containing only one or two entities. We use the F1 score instead of accuracy because accuracy can be dominated by correct predictions for large modules and may hide poor performance on smaller modules. F1 is the harmonic mean of precision and recall. Precision measures the proportion of entities assigned to a module that actually belong to it, whereas recall measures the proportion of entities that truly belong to a module and are correctly assigned.

We report macro- and micro-averaged F1 scores. Macro-F1 is the unweighted average of F1 scores computed separately for each module, giving equal weight to every module regardless of size. Micro-F1 considers all file-module assignments jointly, reflecting overall file-level mapping performance.

V. Results

We report results from 100 runs per model and system. For each run, the same randomly selected seed set initializes the training data for all models, enabling comparisons under identical initial labeling conditions. All metrics are evaluated on the full test set: after training ends, every file receives the prediction of the final trained model, whether or not it was confidently mapped during self-training. This prevents any method from benefiting from a smaller evaluation set. Results are summarized as the median across the 100 runs. This section addresses the three research questions in order.

We assess the statistical significance of performance differences using the Wilcoxon signed-rank test [37] on F1 distributions across 100 runs, pairing runs by their shared random seed across compared configurations. To quantify these differences, we report Cliff's δ [38] as an effect size, classified as negligible ($\delta < 0.147$), small ($0.147 \leq \delta < 0.33$), medium ($0.33 \leq \delta < 0.474$), or large ($\delta \geq 0.474$).

A. RQ1: Overall Effectiveness of C2A Mapping

To answer RQ1, we compare NBA_{CDA}, MLP, GCN, and GAT over 100 runs per system, all initialized from the same labeled set; NBA_{CDA} follows its own published self-training procedure (Section IV-C), while MLP, GCN, and GAT share the iterative learning loop of Section III-D. Table III reports the median macro and micro F1 on ten Java and C/C++ benchmark systems. NBA_{CDA} is included only as a reference from prior work, since all learned models substantially outperform it. The main comparison is therefore between MLP, GCN, and GAT, with MLP isolating the contribution of the node features without exploiting the dependency graph.

GAT achieves the highest micro F1 on nine of the ten systems, either alone or tied, and the highest macro F1 on seven. MLP attains the highest macro F1 on SH-3D, HDF, and HDC, and ties GAT for the highest micro F1 on SH-3D, Lucene, HDF, and HDC. HDF and HDC are among the smallest and least connected benchmark systems, whereas SH-3D is comparatively well connected but exhibits the weakest modular cohesion along with T.Mates; in all three, MLP either outperforms GAT or remains competitive. The benefit of graph-based modeling grows with system size: on Chrome, the only large-scale system in the benchmark, GAT and GCN exceed MLP by 22 and 19 macro-F1 points, and both improve micro F1 by 17 points. GCN consistently outperforms NBA_{CDA} but does not occupy a consistent position between MLP and GAT: it is the lowest of the three on macro F1 for most systems, while on micro F1 it falls between MLP and GAT on only half the benchmark. Section V-B examines what drives these system-level differences.

The C/C++ systems use the same pipeline and typed dependency relations as the Java systems, without language-specific tuning. Bash, HDF, and HDC are also the smallest and sparsest systems in the C/C++ subset, and MLP is competitive with or outperforms the GNNs on two of them.

TABLE III: Median F1 scores ($\times 100$) across 100 runs. MA and MI denote macro- and micro-F1, respectively. The highest score for each system–metric pair is highlighted.

	A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
Method	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
NBA_{CDA}	16	46	13	43	38	70	43	60	24	56	17	52	21	19	31	55	2	37	24	56
MLP	82	90	91	94	66	92	86	95	86	96	91	95	63	83	89	89	59	72	77	88
GCN	79	92	91	94	80	94	85	94	79	93	79	91	66	87	78	83	78	89	70	87
GAT	83	94	92	96	86	96	90	95	85	96	92	97	68	87	82	84	81	89	74	88

Statistical significance of observed differences Table IV reports pairwise statistical comparisons of the three learned methods for the performance differences shown in Table III.

TABLE IV: Median differences in F1 points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor the first method. Cells in which the second method is significantly better than the first are highlighted. † marks a difference that is not statistically significant.

	GAT − GCN		GCN − MLP		GAT − MLP	
System	MA	MI	MA	MI	MA	MI
A.UML	+4.3 (M)	+2.0 (L)	−2.7 (M)	+2.0 (S)	+1.6† (N)	+4.0 (L)
C.Img	+1.0 (M)	+1.5 (L)	−0.3† (N)	−0.1† (N)	+0.7 (S)	+1.4 (M)
JabRef	+6.4 (M)	+1.6 (M)	+13.7 (L)	+2.4 (L)	+20.0 (L)	+3.9 (L)
Lucene	+4.6 (M)	+1.0 (M)	−0.4† (N)	−1.1 (M)	+4.2 (M)	−0.0† (N)
SH-3D	+5.5 (M)	+3.7 (L)	−6.4 (L)	−3.0 (L)	−0.8† (N)	+0.7 (S)
T.Mates	+13.3 (L)	+6.3 (L)	−12.5 (L)	−4.1 (L)	+0.7† (N)	+2.2 (M)
Bash	+2.2† (N)	+0.6† (N)	+3.4† (S)	+3.9 (L)	+5.6 (S)	+4.5 (L)
HDF	+3.9 (S)	+0.8† (N)	−11.3 (L)	−6.5 (L)	−7.4 (L)	−5.7 (L)
Chrome	+2.8 (L)	+0.8 (S)	+18.8 (L)	+16.5 (L)	+21.6 (L)	+17.2 (L)
HDC	+3.9† (S)	+1.1† (N)	−6.4 (M)	−0.4† (N)	−2.5† (S)	+0.7† (N)

GAT significantly outperforms GCN on macro F1 for eight systems and on micro F1 for seven, with mostly medium or large effects; the differences on Bash and HDC are not significant on either metric. GCN's difference from MLP varies substantially by system and by metric. MLP significantly outperforms GCN on macro F1 on A.UML, SH-3D, T.Mates, HDF, and HDC, with medium or large effects, whereas GCN significantly outperforms MLP on JabRef and Chrome on both metrics. On Bash, GCN does not significantly outperform MLP on macro F1 but does on micro F1, with a large effect.

GAT significantly outperforms MLP on macro F1 on C.Img, JabRef, Lucene, Bash, and Chrome, with small to large effects, and on micro F1 on six systems, additionally including A.UML and T.Mates. MLP significantly outperforms GAT on both metrics only on HDF, with large effects. On Bash, GAT significantly outperforms MLP on both metrics despite its low connectivity, an exception addressed in Section V-B. Chrome is the only system with significant large effects on both metrics in all three pairwise comparisons.

How stable is mapping across different initial training sets? Figure 2 shows the distribution of micro F1 scores over 100 runs. The learned methods are generally stable, with interquartile ranges below 6 points on seven of the ten systems. Macro F1 varies more across runs because changes in the initial labeled set disproportionately affect small modules. This variation is largely shared across MLP, GCN, and GAT, suggesting that it is primarily determined by system characteristics rather than model choice. HDC exhibits the greatest variability, with macro-F1 interquartile ranges of 13-14 points, compared with approximately 8 points for micro-F1. MLP produces the narrowest macro-F1 distribution on several systems, including JabRef, SH-3D, T.Mates, Bash, HDF, and HDC, although this pattern is not consistent across all systems.

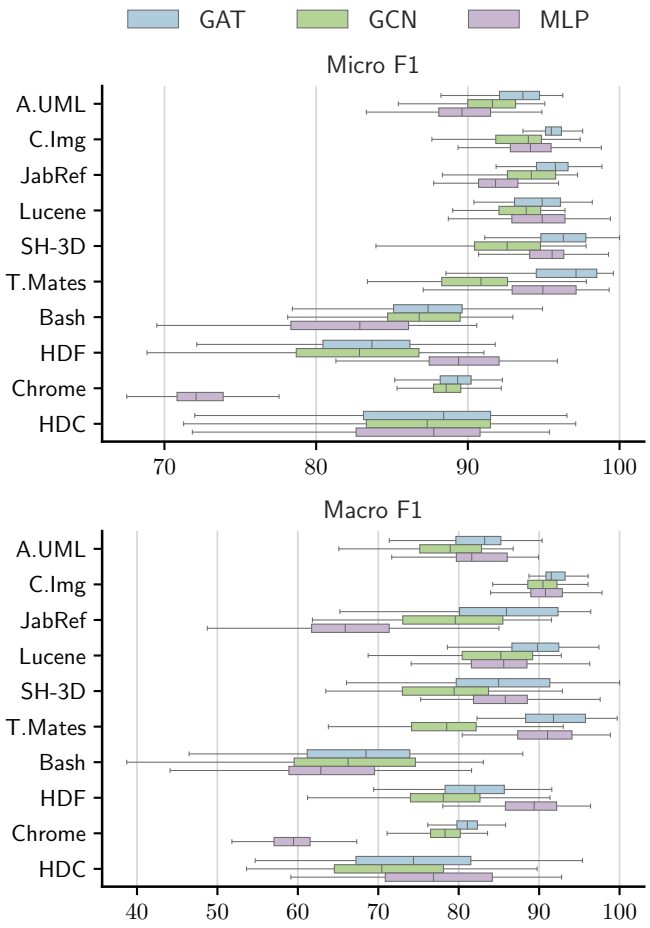


Fig. 2: Distribution of micro F1 (left) and macro F1 (right) scores across 100 runs for each model and system.

Variability also tracks the number of labeled files available at initialization and, in some systems, graph connectivity rather than system size or language. HDC, HDF, Bash, and SH-

3D are the smallest systems and start near the 30-file floor, leaving only one or two labeled files per module. This limited supervision makes performance sensitive to the particular files selected. HDC, HDF, and Bash are also sparse, further limiting the structural signal available to graph-based models, whereas SH-3D is comparatively well connected. C.Img provides a contrast: although small, it is dense and cohesive and remains stable. Once the initial training set reaches roughly 50 files, interquartile ranges generally remain narrow regardless of scale. C.Img and Chrome, the smallest dense system and the largest benchmark system, are among the tightest on macro F1. Residual macro-F1 variability instead tracks module-size skew, as systems with both very large and very small modules, such as JabRef, retain wider macro-F1 distributions despite adequate supervision.

How does the approach scale? To assess scalability, Table V reports the average total runtime of a full pipeline run over five repetitions for each method. A run includes node-feature generation, graph construction for the GNNs, and the complete iterative training procedure. We exclude static-dependency extraction because Depends performs it once per system, and its output is reused by all methods.

TABLE V: Total time consumption (in seconds) averaged over five runs for MLP, GCN, and GAT pipelines.

Dataset	MLP	GCN	GAT
A.UML	12.55	15.29	17.75
C.Img	3.24	5.89	8.24
JabRef	2.60	6.54	8.41
Lucene	9.64	13.95	17.67
SH-3D	19.68	24.14	26.85
T.Mates	17.32	22.35	24.14
Bash	1.09	2.95	4.42
HDF	0.84	4.47	6.90
Chrome	26.93	107.01	123.95
HDC	1.39	3.13	5.41

MLP is fastest on every system because it neither constructs a dependency graph nor performs message passing. GCN adds message passing over typed edges, whereas GAT additionally learns attention weights for edges; this ordering holds across all systems. On Chrome, the largest benchmark system with 18,343 files, a full GAT run completes in under two and a half minutes, while every other system completes in under 40 seconds. SH-3D is the slowest non-Chrome system for every model despite its modest file count, since its files carry substantially more identifiers each, giving Word2Vec a larger corpus to train on relative to the number of files.

B. RQ2: Contribution of the Dependency Graph

RQ1 shows that learned-model performance varies with system characteristics rather than one method dominating throughout. RQ2 examines what the dependency graph and GNNs contribute beyond lexical features: which system properties govern the benefit of message passing, and whether graph-based models yield more consistent file-level assignments across initial training sets, a question median F1 cannot answer since it aggregates performance at the system level. All models use identical lexical features and training loops, so differences between MLP and the GNNs are consistent with a contribution from graph structure and message passing, though the encoders also differ architecturally beyond this.

Answer to RQ1.

GNN-based mapping substantially outperforms NBA_{CDA} on every system. Among the learned methods, GAT achieves the highest mean macro F1 (83.3) and micro F1 (92.2). It attains the highest or tied-highest micro F1 on nine systems and the highest macro F1 on seven. GCN improves over NBA_{CDA} but does not consistently outperform the graph-free MLP baseline. Thus, GAT is the most effective learned approach overall, while the relative performance of the learned methods varies across systems. Runtime remains practical even for Chrome, the largest system in the benchmark, with 18,343 files: a full GAT pipeline run completes in under two and a half minutes. RQ2 examines the contribution of graph structure and message passing beyond lexical features.

Which system properties govern where the graph helps? Two graph properties plausibly govern whether message passing improves upon lexical features alone: connectivity, which determines the volume of neighborhood information available to aggregate, and homophily, which determines whether that information aligns with the file's ground-truth module [39]. Connectivity appears to establish a baseline constraint on the structural signal the graph can provide. Bash, HDC, and HDF have the lowest average node degrees in the benchmark (4.2, 5.4, and 8.0, respectively) and correspond to the cases where GAT performs worst relative to MLP or to its own performance on denser systems. With sparse dependencies, files possess limited local neighborhoods for aggregation, regardless of intra-module cohesion. Furthermore, these systems are the smallest in the benchmark and begin near the 30-file supervision floor, so limited supervision likely compounds the weak structural signal.

Homophily is associated with how GCN and GAT diverge when connectivity is sufficient. On SH-3D ($\mathcal{H} = 0.34$) and T.Mates ($\mathcal{H} = 0.33$), the least homophilic systems in the benchmark, GCN performs significantly worse than MLP with large effect sizes, whereas GAT remains statistically indistinguishable from MLP on both. A plausible explanation is that GCN's fixed degree-normalized aggregation incorporates information from cross-module neighbors, whereas GAT's learned attention mechanism down-weights less informative neighbors and limits the effect of heterophilic dependencies. HDF differs from this pattern. Both GCN and GAT perform significantly worse than MLP with large effect sizes. Its low homophily ($\mathcal{H} = 0.35$), sparse dependency graph, and small initial labeled set together provide too little useful neighbor information for attention to overcome the effect of cross-

module dependencies.

Does message passing make individual-file decisions more repeatable? For each file, we compute its misclassification rate as the proportion of test-set evaluations in which its predicted module differs from the ground-truth module. We refer to files with a misclassification rate of at least 0.9 as *persistently wrong*. Figure 3 shows the distribution of per-file misclassification rates for each model and system, revealing file-level error patterns that aggregate F1 scores can conceal.

For most systems, the cumulative curves for GAT and GCN rise sharply near zero. On A.UML, JabRef, Lucene, and T.Mates, the median file exhibits a misclassification rate near 0 under both GNNs, whereas the MLP median is substantially higher and its distribution extends toward 1. Bash and HDF show why a single summary statistic such as F1 is not enough to judge reliability: MLP is competitive with the GNNs on F1 on these systems, yet its median misclassification rate is 0.98 on Bash and 0.71 on HDF, against medians below 0.15 for GAT and GCN on both. The MLP error distribution on these systems is polarized between a small set of files it always gets right and a much larger set it almost never gets right, while the GNNs concentrate most files at low misclassification rates. The persistently wrong tail confirms this from the opposite end: the share of such files under MLP is large and system-dependent, reaching 78% on Bash and 34% on HDF, while it stays below 5% for both GAT and GCN on every system. This gap in repeatability indicates that message passing contributes beyond raw accuracy. By combining a file's lexical representation with information from its dependency neighborhood, it incorporates structural context that remains fixed across initial training-set draws and can therefore yield more consistent module assignments.

Which files does GAT misclassify persistently? Persistently wrong files (misclassification rate ≥ 0.9) are rare under GAT, at most a few percent of any test set. We examined the dependency neighborhood of the 372 such files GAT produced across all ten systems and found four patterns. The two largest patterns account for 82% of cases and differ in the extent to which the dependency neighborhood aligns with the file's true module.

The dominant pattern, accounting for 60.5% of persistently wrong files, contains files whose neighborhoods are dominated by a module other than their own. Incoming edges exceed outgoing edges for 86% of these files, with median in- and out-degrees of 40 and 6, respectively. Table VI shows nine examples from Chrome, Lucene, JabRef, and A.UML. In each, callers dominate the neighborhood: the true module accounts for at most 4%, whereas the predicted module accounts for at least 55%. These files are interfaces or utilities whose role is defined primarily by the services they provide to the rest of the system rather than the dependencies they use. Message passing therefore receives most of its aggregated information from callers, and when these callers predominantly belong to another module, the resulting representation is biased toward that module. Large modules can supply many callers, but the

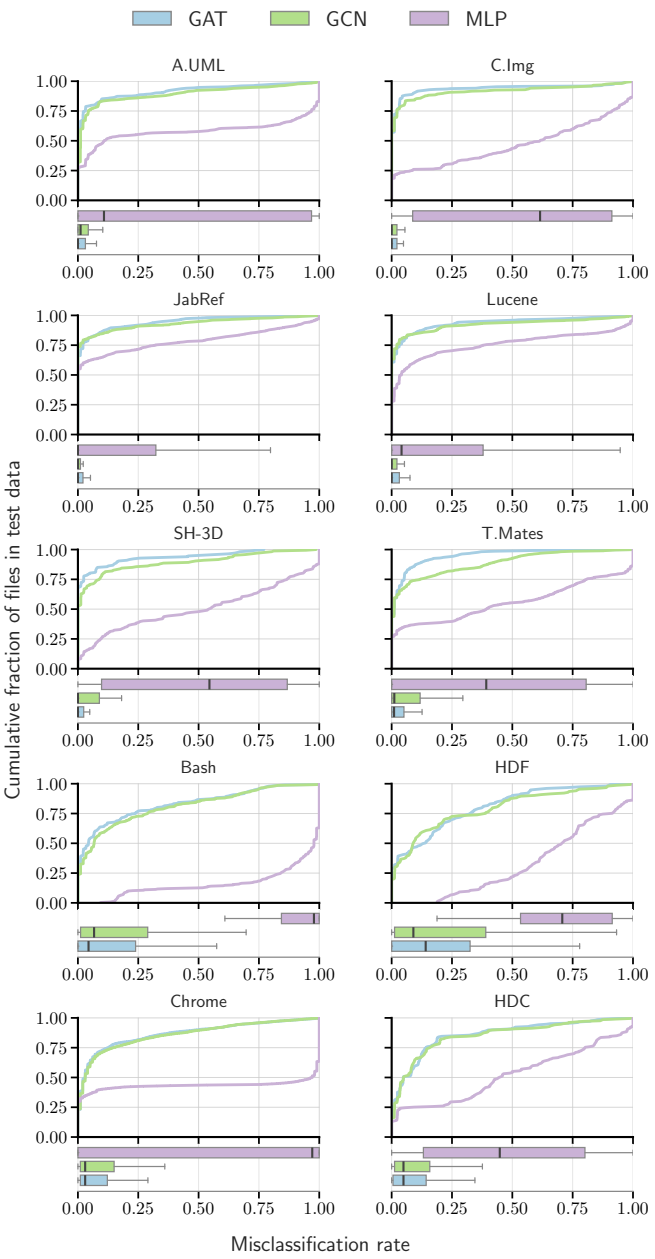


Fig. 3: Empirical cumulative distribution of per-file misclassification rates across 100 runs, one panel per system, for GAT, GCN, and MLP. The y-axis is the cumulative proportion of test-set files whose misclassification rate is at or below the value on the x-axis, so a curve that rises earlier and higher indicates more reliable per-file decisions. The box plot in each panel shows the median, interquartile range, and full range of the misclassification rate for each model.

errors are not explained by module size alone. In every row of Table VI, the predicted module is the module contributing the most callers; it is the largest module in its system in three of the nine cases. Section V-C reports whether reversing or removing edge direction changes F1 on these systems.

The second pattern, accounting for 21.5% of persistently wrong files, consists of files whose neighborhoods are dis-

TABLE VI: Dependency-neighborhood composition for nine files consistently misclassified by GAT in the largest persistent-error pattern, where the neighborhood is dominated by a module other than the true one. GT% is each module's share of files in the system. In/Out are the file's incoming and outgoing dependency edges, de-duplicated by source, target, and dependency type. True/Pred. share is the fraction of the file's neighborhood in its true or predicted module. † marks cases where the predicted module is the system's overall largest.

System	File	True (GT%)	Predicted (GT%)	In	Out	True share	Pred. share
Chrome	v8/src/unbound-queue-inl.h	v8base (1.7%)	webcore_webkit (23.3%)†	896	2	0.003	0.991
	WebCore/platform/text/SuffixTree.h	webcore_platform (4.8%)	webcore_webkit (23.3%)†	12	9	0.000	1.000
	base/allocator/allocator_shim.h	allocator (0.4%)	content (7.3%)	1	0	0.000	1.000
Lucene	document/DateTools.java	document (1.1%)	queryparser (16.5%)	29	2	0.000	0.935
	index/Payload.java	index (19.9%)	analysis (26.0%)	60	4	0.031	0.906
JabRef	JabRefExecutorService.java	logic (60.9%)	gui (29.2%)	48	2	0.040	0.940
	shared/SharedDatabasePreferences.java	preferences (0.9%)	gui (29.2%)	19	10	0.000	0.552
A.UML	swingext/ToolBarUtility.java	swingextensions (1.0%)	diagrams (32.1%)†	21	7	0.000	0.750
	swingext/SpacerPanel.java	swingextensions (1.0%)	critics (8.2%)	35	0	0.000	0.657

tributed across many modules, with no single module dominant. These files have a median of 87 neighbors, but the largest neighboring module accounts for only 40% of the neighborhood. Aggregation therefore provides a diffuse signal that does not strongly support a single module, leaving the prediction more dependent on node features. A third pattern, accounting for 15.6% of persistently wrong files, contains files whose dominant neighboring module matches the ground-truth module but that are nevertheless misclassified. Their neighborhoods are small, with a median of five neighbors, so the correct structural signal is insufficient to overcome lexical features that favor another module. The remaining 2.4% are isolated files with no incoming or outgoing edges. For these files, message passing cannot aggregate neighborhood information, and the prediction depends entirely on node features.

Answer to RQ2.

Two system properties govern the graph's usefulness beyond lexical features: connectivity, which sets a baseline for how much neighborhood information a file can aggregate, and homophily, which determines whether that information supports the file's true module. On the three least-connected systems, GAT underperforms relative to MLP or to its own results on denser systems, except on Bash, where it still significantly outperforms MLP despite low connectivity. GAT outperforms GCN on low-homophily systems, plausibly because attention can down-weight cross-module neighbors. Message passing also produces more repeatable file-level assignments: on most systems the median file has a near-zero misclassification rate under both GNNs, whereas MLP splits between files it almost always gets right and a larger group it almost never does, even when its F1 is comparable. Among persistently wrong files, 82% fall into two neighborhood patterns: callers predominantly from another module, or large but diffuse neighborhoods with no dominant module.

C. RQ3: Sensitivity to Design Choices

RQ1 and RQ2 establish where the approach performs well and what the dependency graph contributes. RQ3 examines whether modeling dependency types separately is necessary and how sensitive the approach is to two message-passing design choices: the number of layers and the direction of aggregation. Because the benchmark systems have highly imbalanced module sizes, macro F1 can be volatile: it gives each module equal weight, so reassigning only a few files in a small module can substantially change the score. Micro F1 instead weights each file equally and may remain largely unchanged by the same reassignments. We therefore report both metrics throughout and regard a significant difference in either metric as evidence that the design choice affects performance.

Does modeling dependency types separately matter? Our encoders keep one adjacency matrix per relation type and learn a separate transformation for each, rather than collapsing all dependencies into a single edge type. Figure 4 compares this multi-relational design against a uni-relational variant, in which all dependency types are merged into one undifferentiated adjacency matrix before message passing, for both GAT and GCN across the ten benchmark systems.

Multi-relational aggregation outperforms the uni-relational variant on every system for both GAT and GCN, with median macro F1 gains of 6.7 to 36.6 points for GAT and 7.4 to 32.3 points for GCN. The smallest gain occurs on C.Img for both encoders, and the largest on T.Mates. This is consistent with different dependency types carrying different structural signals about module membership: a call edge and an inheritance edge do not imply the same likelihood that two files share a module. The multi-relational encoder also has more parameters than the uni-relational, so part of the gain may reflect this added capacity rather than the relation typing alone.

How many message-passing layers does the graph support? Additional GNN layers enlarge each node's receptive field, which is the L-hop neighborhood that can influence its representation after L message-passing layers. However, when

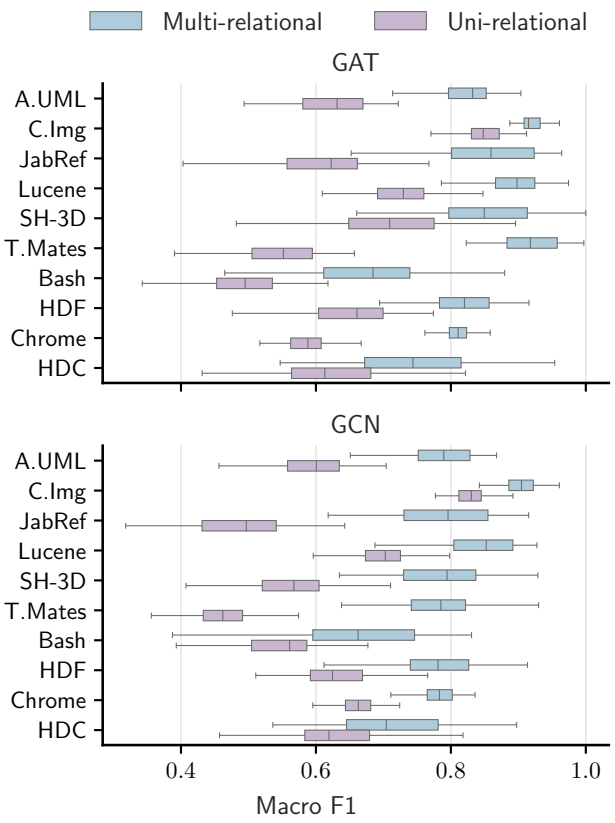


Fig. 4: Distribution of macro F1 across 100 runs for the multi-relational and uni-relational variants of GAT and GCN, one panel per model, across all ten systems.

repeated aggregation mixes information from a large portion of the graph, node representations can become increasingly similar and lose the discriminative information needed for classification, an effect known as over-smoothing [40]. We therefore explore only one and two layers, and Table VII reports the comparison. For GAT, two layers significantly improve JabRef and Chrome, and significantly hurt only HDC, on macro F1; elsewhere the difference is not significant. GCN shows the opposite trend: significant gains appear only on JabRef and Bash, on both metrics, and on Chrome by micro F1, while every other system shows a significant loss or no significant gain on at least one metric. Unlike GAT, GCN uses fixed degree-based weights and cannot assign lower weights to individual neighbors based on whether their features are informative for module prediction, so a second layer helps only when the two-hop neighborhood stays sufficiently aligned with the target file's module. JabRef shows the clearest case: its few, large modules keep most two-hop neighbors within the same module, and it is where GCN gains the most. Bash is a partial exception to the connectivity pattern: despite having the lowest average degree in the benchmark, its high homophily ($\mathcal{H} = 0.60$) keeps the few two-hop neighbors it does reach within the same module, so the second layer still helps. T.Mates shows the opposite: many neighbors belong to other modules, so the second layer introduces more cross-module information and produces the largest loss after HDC. HDF, SH-3D, and HDC add a further constraint: too few files for a second hop to add meaningfully different context, so depth adds noise regardless of module composition. Overall, two layers are a safe default for GAT, whereas a single layer is safer for GCN outside large, coarse-grained systems such as JabRef.

TABLE VII: Median differences in F1 points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor two layers; cells in which one layer is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT (2L − 1L)		GCN (2L − 1L)	
System	MA	MI	MA	MI
A.UML	$+0.2^{\dagger}$ (N)	$+0.1^{\dagger}$ (N)	$-0.2^{\dagger}$ (N)	$-0.5^{\dagger}$ (N)
C.Img	$+0.1^{\dagger}$ (N)	$+0.0^{\dagger}$ (N)	$-0.6^{\dagger}$ (N)	-0.8 (S)
JabRef	$+5.2$ (M)	$+1.0$ (M)	$+4.6$ (M)	$+1.6$ (M)
Lucene	$+1.8^{\dagger}$ (S)	-1.1 (S)	$+2.8^{\dagger}$ (S)	$-0.3^{\dagger}$ (N)
SH-3D	$+1.7^{\dagger}$ (S)	$+1.4$ (S)	$+0.3^{\dagger}$ (N)	$-1.5^{\dagger}$ (S)
T.Mates	$+1.5^{\dagger}$ (S)	$+0.3^{\dagger}$ (N)	-7.3 (L)	-3.1 (L)
Bash	$+4.2^{\dagger}$ (S)	$+3.7$ (M)	$+3.9$ (S)	$+2.4$ (M)
HDF	$+0.2^{\dagger}$ (N)	$-0.3^{\dagger}$ (N)	$-0.7^{\dagger}$ (N)	$+1.0^{\dagger}$ (S)
Chrome	$+2.9$ (L)	$+2.8$ (L)	$+0.1^{\dagger}$ (N)	$+1.3$ (L)
HDC	-5.2 (S)	$-0.9^{\dagger}$ (N)	-8.9 (M)	$-1.8^{\dagger}$ (S)

Does edge direction matter? The heterogeneous encoder aggregates source-to-target by default, so a file's representation is shaped by the neighbors that depend on it rather than by the neighbors it depends on. Table VIII compares this against an undirected variant, which symmetrizes every edge, and a reverse variant, which aggregates target-to-source. For GAT, source-to-target aggregation significantly outperforms the reversed variant on six of ten systems, with the largest gains on SH-3D and HDC, and significantly underperforms only on Lucene. GCN shows no comparable consistency: it significantly outperforms the alternatives on five systems, most notably reversed aggregation, but significantly underperforms on four, A.UML, Lucene, Bash, and HDF. RQ2 showed that persistently misclassified files often have caller-dominated neighborhoods from another module. Chrome and JabRef show that reversing or symmetrizing edges does not resolve this pattern: both alternatives remain significantly worse than the directed default. Lucene is an exception: reversed aggregation significantly improves performance for both encoders. Its high homophily ($\mathcal{H} = 0.68$), the highest in the benchmark, indicates that dependency edges more often connect files from the same module, which may reduce the harm of aggregating in the reverse direction.

VI. Threats to Validity

External Validity: The benchmark systems, spanning Java, C, and C++, are not a representative sample of architectural styles, application domains, or team practices; they were selected because a ground-truth mapping was available and usable for our evaluation. Chrome is the only system at

TABLE VIII: Median F1-point differences for macro-F1 (MA) and micro-F1 (MI) under the default directed (D) source-to-target aggregation, compared with two alternatives: UD (undirected, symmetrized edges) and RD (reversed, target-to-source), with Cliff's δ magnitude shown in parentheses (N/S/M/L). Positive values favor D, and cells where the alternative is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT				GCN			
	D − UD		D − RD		D − UD		D − RD	
System	MA	MI	MA	MI	MA	MI	MA	MI
A.UML	$+2.1^{\dagger}$ (S)	+1.3 (M)	$-1.7^{\dagger}$ (S)	$-0.3^{\dagger}$ (N)	−2.3 (S)	−1.3 (M)	−4.2 (M)	−1.6 (M)
C.Img	+2.7 (L)	+0.9 (M)	+5.5 (L)	$+0.4^{\dagger}$ (S)	+2.9 (M)	$+0.3^{\dagger}$ (N)	+8.7 (L)	$-0.5^{\dagger}$ (N)
JabRef	+10.0 (L)	+1.2 (M)	+10.1 (L)	$+0.0^{\dagger}$ (N)	$+2.9^{\dagger}$ (N)	$-0.3^{\dagger}$ (N)	+8.7 (L)	$-0.8^{\dagger}$ (N)
Lucene	+2.8 (S)	$+1.0^{\dagger}$ (S)	−4.3 (L)	−1.4 (M)	$-2.6^{\dagger}$ (S)	$-0.5^{\dagger}$ (S)	−8.0 (L)	−1.5 (S)
SH-3D	+16.0 (L)	+7.4 (L)	+15.3 (L)	+5.9 (L)	+10.0 (L)	+3.7 (L)	+17.5 (L)	+5.9 (L)
T.Mates	+7.2 (L)	$+1.2^{\dagger}$ (S)	+7.1 (L)	$-0.1^{\dagger}$ (N)	$-0.7^{\dagger}$ (S)	−2.6 (M)	$-0.3^{\dagger}$ (N)	−5.0 (L)
Bash	$-2.3^{\dagger}$ (S)	$-0.6^{\dagger}$ (N)	$-1.8^{\dagger}$ (S)	$-0.6^{\dagger}$ (N)	$-3.9^{\dagger}$ (S)	$-1.1^{\dagger}$ (N)	−4.2 (S)	$-1.1^{\dagger}$ (N)
HDF	+5.1 (M)	+4.8 (L)	$-2.0^{\dagger}$ (S)	$-0.8^{\dagger}$ (N)	$+1.4^{\dagger}$ (S)	+4.9 (M)	−3.8 (S)	$-0.1^{\dagger}$ (N)
Chrome	+6.4 (L)	+2.0 (L)	+5.5 (L)	$+0.4^{\dagger}$ (N)	+2.4 (L)	$+0.1^{\dagger}$ (N)	+3.3 (L)	$+0.3^{\dagger}$ (N)
HDC	+8.4 (M)	$+0.6^{\dagger}$ (N)	+10.7 (L)	$+1.0^{\dagger}$ (N)	$+0.9^{\dagger}$ (N)	$-0.9^{\dagger}$ (N)	+10.7 (L)	$+0.5^{\dagger}$ (N)

Answer to RQ3.

Multi-relational aggregation beats a single merged edge type on every system for both encoders, since each dependency type carries a distinct signal about module membership. GAT is comparatively robust to depth and direction, with two layers and source-to-target aggregation providing reasonable defaults across systems. GCN is more sensitive to both settings, and the appropriate choice depends on system properties: coarse module granularity favors two layers, whereas low homophily or weak connectivity favors one. Neither setting resolves the persistently misclassified files identified in RQ2, motivating future work on weighting a file's own features against its neighborhood and on extending the approach to dynamically typed languages, where dependency extraction is less precise.

industrial scale, so what we report about scaling rests on one observation rather than a trend. None of the systems consist largely of AI-generated code, and whether such code produces dependency structures with comparable module cohesion is untested.

Our approach is transductive and assumes a fixed graph during training. New files or dependencies require graph reconstruction and retraining, making the method more suitable for periodic analyses and maintenance workflows than for incremental updates at commit time.

Internal Validity: The dependencies are extracted by a static analyzer that resolves references syntactically. It may miss dependencies arising through runtime, such as reflection and function pointers, as well as dependencies introduced by conditional compilation or macro expansion. It may also introduce edges that do not represent actual dependencies when it cannot resolve a reference precisely. Both error types affect the GNNs directly, since extraction mistakes translate into missing or incorrect edges that message passing relies on.

A separate source of error comes from matching the version of the source code we analyze to the version of the underlying published ground-truth mapping. Several benchmarks were mapped years before we obtained the code, and some files only exist depending on how the system was built, so the exact set of files can differ between the mapping and our study.

Files that plausibly belong to two modules may be assigned differently by different annotators, and older systems in the benchmark may have accumulated architectural drift that weakens the cohesion between a file and its assigned module. Because the initial labeled set is drawn from the same mapping used for evaluation, an annotation error can both mislead training and penalize a correct prediction.

Our primary unit of module assignment and analysis is the file, following the granularity of the available ground truth. A file implementing more than one concern must nevertheless receive a single label, which none of the compared models can express.

Comparison against the NBA required a Python reimplementation extended to C/C++, since the original targets Java. We preserved the published procedure and supplied it with the same labeled files given to the other models, though we cannot fully exclude divergence from the original code.

The statistical comparisons draw on the same ten benchmark systems throughout, so the reported effect sizes and significance patterns reflect this specific set of systems and may not generalize to systems outside it.

Construct Validity: A fixed confidence threshold determines whether a file receives a pseudo-label, which reflects how sharply a model concentrates its softmax output and not only how well it maps files. Reporting scores over the complete test set prevents a model that maps fewer files from benefiting from an easier evaluation set.

Directory paths are part of the node features, and in several benchmark systems the folder structure closely reflects the intended module decomposition. Where this alignment holds, reported scores may be optimistic, since the model has partial access to the answer through the path feature itself rather than recovering the mapping from identifiers or dependency

structure alone.

Timing covers feature generation, graph construction, and training on one hardware configuration, so the absolute figures are specific to that setup, and only the relative ordering across models and systems transfers.

VII. Conclusion

We studied code-to-architecture mapping as semi-supervised node classification over a directed multi-relational dependency graph, comparing GAT and GCN against an MLP baseline that receives the same lexical features and passes through the same self-training loop, so that differences are consistent with the contribution of the dependency structure and message passing. Across ten systems in Java and C/C++ and 100 runs per system, all three learned models improve on the Naive Bayes reference by a wide margin, and GAT reaches the highest average macro F1, ahead of MLP and GCN, though MLP remains competitive with or ahead of both GNNs on the smallest and least connected systems, as well as those with low intra-module cohesion, in the benchmark. The advantage of the graph grows rather than shrinks with scale: on Chrome, the largest system in the benchmark at 18,343 files, GAT and GCN outperform MLP by 22 and 19 macro F1 points.

Two system properties govern where this advantage appears: connectivity sets a baseline for how much neighborhood information a file can aggregate, and homophily determines whether that information supports the file's true module. Message passing also makes individual file-level decisions substantially more repeatable across different initial training sets: the share of persistently misclassified files stays below 5% for both GAT and GCN on every system, against as much as 78% for MLP on the sparsest systems, even where MLP's aggregate F1 is comparable.

Modeling dependency types separately matters, as different edge types carry different structural signals, and letting the encoder learn a separate transformation per relation outperforms a single merged edge type on every system. Two layers and source-to-target aggregation are reasonable defaults for GAT, while GCN's optimal depth and direction depend on system granularity, connectivity, and homophily. A full run completes in under 40 seconds on every system except Chrome, which takes under two and a half minutes, so scale is not what limits the approach.

The limitation of the approach is not scalability but the quality of the available structural signal. Message passing cannot fully resolve cases in which a file's dependency neighborhood contradicts its true module or provides too little evidence for reliable classification. Three directions could address this limitation. First, relation-aware attention could allow GAT to weight dependency types differently when they provide conflicting evidence, rather than deriving attention solely from node features. Second, the graph could incorporate finer-grained architectural information, including folder hierarchies and class members, to better reflect how architects conceptualize modules. Third, further isolating the contribution of lexical features would clarify how much architectural signal they provide independently of the dependency graph.

References

[1] P. Clements, D. Garlan, R. Little, R. Nord, and J. Stafford, "Documenting software architectures: views and beyond," in *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 2003, pp. 740–741.

[2] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012. [Online]. Available: https://philippe.kruchten.com/wp-content/uploads/2012/08/kruchten-120821-techdebt.pdf

[3] R. Ehsani, A. Aljohani, H. Do, and Y. Cai, "Faster code, deeper debt? A multivocal literature review on technical debt and its early signs in LLM-assisted software development," *ACM Transactions on Software Engineering and Methodology*, 2025.

[4] W. Wu, H. Hu, Z. Fan, Y. Qiao, Y. Huang, Y. Li, Z. Zheng, and M. R. Lyu, "An empirical study of code clones from commercial AI code generators," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 2874–2896, 2025.

[5] L. De Silva and D. Balasubramaniam, "Controlling software architecture erosion: A survey," *Journal of Systems and Software*, vol. 85, no. 1, pp. 132–151, 2012.

[6] G. C. Murphy, D. Notkin, and K. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *3rd ACM SIGSOFT Symposium on Foundations of Software Engineering*, 1995, pp. 18–28.

[7] G. C. Murphy, D. Notkin, and K. J. Sullivan, "Software reflexion models: Bridging the gap between design and implementation," *IEEE Transactions on Software Engineering*, vol. 27, no. 4, pp. 364–380, 2001.

[8] N. Ali, S. Baker, R. O'Crowley, S. Herold, and J. Buckley, "Architecture consistency: State of the practice, challenges and requirements," *Empirical Software Engineering*, vol. 23, pp. 224–258, 2018.

[9] J. Garcia, I. Krka, C. Mattmann, and N. Medvidovic, "Obtaining ground-truth software architectures," in *35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 901–910.

[10] A. Christl, R. Koschke, and M.-A. Storey, "Automated clustering to support the reflexion method," *Information and Software Technology*, vol. 49, no. 3, pp. 255–274, 2007.

[11] R. A. Bittencourt, G. J. d. Santos, D. D. S. Guerrero, and G. C. Murphy, "Improving automated mapping in reflexion models using information retrieval techniques," in *17th Working Conference on Reverse Engineering*, 2010, pp. 163–172.

[12] T. Olsson, M. Ericsson, and A. Wingkvist, "To automatically map source code entities to architectural modules with naive bayes," *Journal of Systems and Software*, vol. 183, p. 111095, 2022.

[13] Z. T. Sinkala and S. Herold, "Inmap: Automated interactive code-to-architecture mapping recommendations," in *18th International Conference on Software Architecture*, 2021, pp. 173–183.

[14] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "GAER: Graph auto-encoders for unsupervised software architecture recovery," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, Jul. 2026.

[15] ——, "Graph representation learning for software architecture recovery." in *ESANN*, 2026.

[16] D. Amalfitano, M. De Luca, T. Santilli, P. Pelliccione, and A. R. Fasolino, "Automated software architecture design recovery from source code using llms," in *European Conference on Software Architecture*. Springer, 2025, pp. 73–89.

[17] R. Pan, B. Mao, T. Ma, and Z. Ling, "ArchAgent: Scalable legacy software architecture recovery with LLMs," in *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026.

[18] D. Fuchß, H. Liu, T. Hey, J. Keim, and A. Koziolek, "Exarch: Enabling architecture traceability by llm-based architecture component name extraction," in *SE2026*. Gesellschaft für Informatik, Bonn, 2026.

[19] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "Graph convolutional networks for mapping source code entities to architectural modules," in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2025, pp. 472–481.

[20] V. Tzerpos and R. C. Holt, "The orphan adoption problem in architecture maintenance," in *Working Conference on Reverse Engineering*. IEEE, 1997, pp. 76–82.

[21] A. Florean, L. Jalal, Z. T. Sinkala, and S. Herold, "A comparison of machine learning-based text classifiers for mapping source code

to architectural modules," in *15th European Conference on Software Architecture*, 2021.
[22] S. A. Rukmono, L. Ochoa, and M. Chaudron, "Deductive software architecture recovery via chain-of-thought prompting," in *44th International Conference on Software Engineering: New Ideas and Emerging Results*, 2024, pp. 92–96.
[23] N. Johansson, M. Caporuscio, and T. Olsson, "Mapping source code to software architecture by leveraging large language models," in *European Conference on Software Architecture*. Springer, 2024, pp. 133–149.
[24] N. Johansson and M. Caporuscio, "Mapping source code to software architecture using embedding models: Insights from an analysis of data properties," *SSRN Electronic Journal*, 2025, preprint, SSRN 5798927. [Online]. Available: https://ssrn.com/abstract=5798927
[25] J. Chen, Z. Wang, Y. Jiang, J. Pang, T. Zhang, M. Pan, and J. Sun, "NEGAR: network embedding guided architecture recovery for software systems," in *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2022, pp. 367–376.
[26] Y. Zhang, Z. Xu, C. Liu, H. Chen, J. Sun, D. Qiu, and Y. Liu, "Software architecture recovery with information fusion," in *Proceedings of the 31st ACM Joint ESEC/FSE*, 2023, pp. 1535–1547.
[27] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *Advances in Neural Information Processing Systems*, vol. 26, 2013.
[28] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "Graph embeddings for software architecture recovery," *Nordic Machine Intelligence*, 2026. [Online]. Available: https://journals.uio.no/NMI/article/view/13153
[29] I. Stavropoulou, M. Grigoriou, and K. Kontogiannis, "Case study on which relations to use for clustering-based software architecture recovery," *Empirical Software Engineering*, vol. 22, no. 4, pp. 1717–1762, 2017.
[30] G. Zhang and W. Jin, "Depends: A fast, comprehensive code dependency analysis tool," https://github.com/multilang-depends/depends, 2018, software repository, accessed: 2026-08-28.
[31] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
[32] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th international conference on machine learning (ICML-10)*, 2010, pp. 807–814.
[33] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.
[34] J. Brunet, R. A. Bittencourt, D. Serey, and J. Figueiredo, "On the evolutionary nature of architectural violations," in *2012 19th Working Conference on Reverse Engineering*. IEEE, 2012, pp. 257–266.
[35] J. Lenhard, M. Blom, and S. Herold, "Exploring the suitability of source code metrics for indicating architectural inconsistencies," *Software Quality Journal*, vol. 27, no. 1, pp. 241–274, 2019.
[36] T. Lutellier, D. Chollak, J. Garcia, L. Tan, D. Rayside, N. Medvidović, and R. Kroeger, "Measuring the impact of code dependencies on software architecture recovery techniques," *IEEE Transactions on Software Engineering*, vol. 44, no. 2, pp. 159–181, 2017.
[37] F. Wilcoxon, "Individual comparisons by ranking methods," in *Breakthroughs in statistics: Methodology and distribution*. Springer, 1992, pp. 196–202.
[38] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions." *Psychological bulletin*, vol. 114, no. 3, p. 494, 1993.
[39] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," *Advances in neural information processing systems*, vol. 33, pp. 7793–7804, 2020.
[40] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.